Guía Estratégica y Arquitectura Técnica para el Lanzamiento Comercial de SaaS Agéntico: Orquestación mediante Google Antigravity

El desarrollo de software a nivel global ha experimentado una transición tectónica, abandonando de manera definitiva los flujos de trabajo tradicionales impulsados por herramientas estáticas para adoptar sistemas orquestados por agentes de inteligencia artificial con capacidad de razonamiento, acción y colaboración autónoma.[1] Esta evolución paradigmática redefine el rol del desarrollador, quien pasa de ser un codificador manual de líneas de texto a convertirse en un arquitecto y gerente de una fuerza laboral digital.[2] En el contexto específico de la creación, maduración y lanzamiento de una plataforma de Software as a Service (SaaS) centrada en la automatización mediante inteligencia artificial en el mercado argentino, la convergencia de estrategias comerciales sólidas, marcos regulatorios locales y entornos de desarrollo agéntico avanzados resulta fundamental.

El presente informe detalla un análisis exhaustivo y un plan de acción riguroso, paso a paso, para llevar un proyecto tecnológico desde su fase conceptual y de desarrollo local hasta su comercialización exitosa y despliegue en producción. Este documento utiliza como núcleo operativo la plataforma Google Antigravity, estableciendo la metodología exacta para inyectar cada componente del negocio (legal, financiero, de producto y de infraestructura) en el entorno de desarrollo impulsado por inteligencia artificial.

Fase 1: Análisis de Mercado, Producto Mínimo Viable y Estrategia de Lanzamiento

Antes de proceder con la inyección técnica de código y lógica de negocio en el entorno de desarrollo, el proyecto debe ser conceptualizado estrictamente bajo la doctrina del Producto Mínimo Viable (MVP) y la validación comercial temprana.[3] El lanzamiento de un SaaS en un ecosistema dinámico y económicamente complejo como el argentino exige una planificación precisa que evite el sobredesarrollo inicial, mitigue los riesgos financieros y priorice la validación iterativa con usuarios reales.[3]

El Ecosistema de las PyMEs Argentinas y la Demanda de Automatización

El ecosistema empresarial local, compuesto mayoritariamente por pequeñas y medianas empresas (PyMEs), presenta una demanda crítica y creciente de soluciones de inteligencia

artificial que sean operativamente accesibles y demuestren un impacto inmediato en la estructura de costos.[5] La literatura académica y los programas de formación ejecutiva, como las diplomaturas en Inteligencia Artificial Práctica dictadas por instituciones como la Universidad de Belgrano, evidencian que el mercado ha madurado más allá de la simple curiosidad tecnológica para buscar herramientas concretas de toma de decisiones y sistemas autónomos escalables.[6]

Las aplicaciones de mayor tracción comercial en este segmento no requieren la construcción de modelos fundacionales desde cero, sino la orquestación inteligente de flujos de trabajo existentes para resolver fricciones operativas diarias. La automatización de tareas repetitivas, como la clasificación semántica de correos electrónicos, la generación de reportes ejecutivos interactivos a partir de archivos de valores separados por comas (CSV) y la atención al cliente mediante chatbots conversacionales que operan de manera ininterrumpida, representan las áreas de mayor valor añadido.[5] La implementación de un chatbot de atención al cliente, por ejemplo, ha demostrado empíricamente la capacidad de reducir los tiempos de respuesta de horas a fracciones de segundo, generando un ahorro operativo cuantificable de entre cuatro y seis horas diarias de labor humana en una PyME promedio.[5]

Del mismo modo, la integración de herramientas de orquestación de flujos de trabajo como Make o n8n, en conjunción con interfaces de programación de aplicaciones (APIs) de modelos de lenguaje, permite la eliminación de hasta el ochenta por ciento de las tareas manuales de ingreso y procesamiento de datos.[5] Para justificar la adopción del SaaS frente a estos clientes potenciales, la propuesta de valor debe articularse de manera inequívoca en torno al Retorno de Inversión (ROI). La eficiencia de la automatización puede modelarse matemáticamente para demostrar el impacto directo en el flujo de caja del cliente. El cálculo fundamental del retorno se expresa mediante la siguiente ecuación estándar, la cual debe ser el núcleo del mensaje de marketing:

$$ROI = \left(\frac{\text{Valor del Tiempo Ahorrado} + \text{Incremento de Ingresos} - \text{Costo del SaaS}}{\text{Costo del SaaS}} \right) \times 100$$

El Plan de Acción de 30/60/90 Días y el Lanzamiento Silencioso

La penetración de mercado debe estructurarse mediante un plan operativo de noventa días que garantice un despliegue fluido y un retorno de inversión sostenido.[9] Los primeros treinta días deben enfocarse en la identificación de los cuellos de botella y procesos críticos del cliente piloto, evitando el error común de intentar automatizar flujos de trabajo que no se comprenden a cabalidad.[9] Los siguientes treinta días se destinan a la asignación de recursos y la medición de avances iniciales, culminando en los últimos treinta días con la consolidación de las métricas de éxito, tales como la reducción de errores humanos y el aumento de la facturación.[9]

El diseño del MVP debe centrarse exclusivamente en maximizar esta métrica para el usuario

final, evitando la saturación de funcionalidades secundarias. El proceso de salida al mercado (Go-To-Market) no debe consistir en un evento masivo inicial, sino que debe contemplar una fase preliminar denominada lanzamiento silencioso (silent launch).[4] Esta estrategia consiste en liberar la plataforma a un grupo reducido y controlado de usuarios antes de someterla al escrutinio del tráfico masivo.[4] La lógica detrás de esta decisión radica en que plataformas de agregación y lanzamiento otorgan un impulso algorítmico significativo, pero de corta duración; llegar a estos picos de tráfico con un producto que presenta errores no resueltos (bugs) o una interfaz de usuario confusa resulta en una pérdida irreparable de clientes potenciales.[4]

El lanzamiento silencioso funciona como una versión alfa en un entorno de producción, permitiendo al equipo de desarrollo ajustar los textos publicitarios (copies), refinar la interfaz de usuario (UI) y, si el tiempo lo permite, integrar aquella funcionalidad específica que los primeros usuarios demandan abrumadoramente.[4] Casos de estudio recientes en la industria, como el despliegue del producto Combo Hunter, demuestran que llegar preparado con un producto rodado, testimonios reales (testimonial selling) y un mensaje validado permite capitalizar verdaderamente el flujo constante de tráfico gratuito que proveen las plataformas de lanzamiento.[4] La analítica de producto desde el primer día es innegociable para comprender las tasas de retención, la activación de usuarios y el valor del ciclo de vida del cliente (CLV), garantizando así un crecimiento sostenido, predecible y escalable.[10]

Fase 2: Arquitectura Regulatoria, Impositiva y de Facturación en Argentina

La viabilidad comercial a mediano y largo plazo de un proyecto SaaS en Argentina está intrínsecamente ligada a su correcta estructuración fiscal, legal y financiera desde las etapas más tempranas de su concepción. El sistema tributario nacional, administrado y fiscalizado por la Administración Federal de Ingresos Públicos (AFIP), rebautizada transitoriamente en discursos recientes como Arca, impone normativas estrictas de trazabilidad electrónica que el software debe integrar de manera nativa para operar en estricto cumplimiento legal.[12]

Categorización Fiscal: Monotributo versus Responsable Inscripto

La primera decisión estratégica estructural radica en la categorización fiscal de la entidad operadora del SaaS, debiendo elegir entre el Régimen Simplificado para Pequeños Contribuyentes (Monotributo) y el Régimen General (Responsable Inscripto).[12] El Monotributo ofrece una simplicidad administrativa innegable, consolidando el componente impositivo, previsional y de obra social en una cuota fija mensual, lo cual resulta ideal para la fase de lanzamiento silencioso y validación inicial.[12] Sin embargo, sus límites de facturación máxima anual restringen severamente la escalabilidad inherente a un modelo de negocio de suscripciones de software exitoso.

La transición hacia la figura de Responsable Inscripto resulta inevitable en fases de crecimiento

acelerado. Esta migración introduce complejidades operativas sustanciales en la liquidación del Impuesto al Valor Agregado (IVA) y el Impuesto a las Ganancias.[12] Mientras que en la exportación de servicios el impacto del IVA puede neutralizarse, en la prestación de servicios a empresas locales, el profesional o la empresa desarrolladora enfrenta un escenario donde los márgenes de ganancia bruta pueden parecer mayores, pero la carga tributaria sobre la ganancia neta aumenta de manera exponencial.[12]

El esquema de tributación para Responsables Inscriptos requiere una atención algorítmica particular debido a su estructura de alícuotas progresivas. La rentabilidad neta proyectada del SaaS debe calcularse absorbiendo o trasladando estas cargas impositivas al usuario final en la estrategia de precios (pricing). La siguiente tabla ilustra la estructura representativa de tramos impositivos aplicables para la liquidación del impuesto, detallando la tasa marginal impuesta sobre el excedente de la ganancia neta sujeta a impuesto:

Límite Inferior de Ganancia Neta (Desde)	**Límite Superior de Ganancia Neta (Hasta)**	**Alícuota Marginal Aplicable sobre el Excedente**
$0	$2.000.030	5%
$2.000.030	$4.000.060	9%
$4.000.060	$6.000.090	12%
$6.000.090	$9.000.135	15%
$9.000.135	$18.000.271	19%
$18.000.271	$27.000.406	23%
$27.000.406	$40.500.609	27%

Orquestación de la Facturación Electrónica AFIP

El motor de facturación de la plataforma SaaS, que deberá ser inyectado posteriormente en el código mediante agentes de IA, debe estar programado con precisión milimétrica para emitir los comprobantes electrónicos correspondientes según la naturaleza de la transacción comercial y la condición fiscal del receptor del servicio.

Las operaciones comerciales realizadas (B2B) entre dos sujetos categorizados como Responsables Inscriptos, o de un Responsable Inscripto a un Monotributista, requieren obligatoriamente la emisión de comprobantes Clase "A".[13] Estos comprobantes no son estáticos; dependiendo del perfil de riesgo y la autorización del emisor, pueden requerir la inclusión de leyendas específicas impuestas por la autoridad fiscal, tales como "Pago en CBU informada" u "Operación sujeta a retención".[13] Es un imperativo técnico programar la lógica condicional del SaaS para que, al procesar comprobantes con la leyenda de operación sujeta a retención recibidos de proveedores, se apliquen las detracciones financieras adecuadas. Estas retenciones, cuando no aplican a receptores monotributistas, ascienden típicamente al cien por ciento de la alícuota del Impuesto al Valor Agregado y al seis por ciento en el caso del impuesto a las ganancias sobre el importe neto determinado.[13]

Por otro lado, las transacciones destinadas a consumidores finales, sujetos exentos en el gravamen, sujetos no alcanzados o turistas extranjeros demandan la emisión de comprobantes Clase "B".[13] Dado que el paradigma del software como servicio posee una vocación de alcance global intrínseca, la captación de clientes internacionales es un vector de crecimiento crucial para diversificar el riesgo país. Para las operaciones transfronterizas consideradas como exportación de servicios, la plataforma debe automatizar inexcusablemente la generación de comprobantes Clase "E", los cuales están exentos del recargo del IVA pero requieren el ingreso de divisas a través del Mercado Único y Libre de Cambios (MULC), sujeto a las normativas del Banco Central.[13]

Simultáneamente a la arquitectura fiscal, el proyecto tecnológico debe alinear su tratamiento de información con las leyes nacionales de protección de datos personales vigentes en Argentina. Además, debe adherirse a las directrices establecidas en el documento de Recomendaciones para el uso de Inteligencia Artificial emitido por el Estado Nacional, asegurando la transparencia algorítmica, la explicabilidad de las decisiones automatizadas y la mitigación activa de sesgos discriminatorios en los modelos que la plataforma ofrezca a sus clientes.[15]

Fase 3: Arquitectura de Cobros y Gestión de Suscripciones Recurrentes

La monetización sostenida de un modelo SaaS depende íntegramente de una infraestructura

de pagos recurrentes que sea robusta, tolerante a fallos de red y que esté sociológicamente optimizada para el contexto local.[16] En la República Argentina, la integración técnica con pasarelas de pago consolidadas, primariamente Mercado Pago, representa un requisito fundamental para maximizar las tasas de conversión.[16] Las fricciones inherentes al uso de tarjetas de crédito internacionales, las limitaciones en los cupos de moneda extranjera y la profunda penetración de las billeteras virtuales locales hacen que prescindir de estas herramientas sea comercialmente inviable.

Estructuración de la Propuesta de Valor por Niveles (Tiered Pricing)

La estrategia comercial exige la definición clara, desde la base de datos hasta la interfaz gráfica, de niveles de suscripción (tiers) diseñados meticulosamente para capturar valor a lo largo de diferentes segmentos de mercado. Esta arquitectura de precios debe fundamentarse en el análisis de la elasticidad precio de la demanda del producto, permitiendo optimizar los ingresos netos y la adopción masiva simultáneamente.[16]

Un modelo de precios por niveles típicamente incluye configuraciones básicas destinadas a usuarios individuales o profesionales independientes, configuraciones intermedias o profesionales para equipos emergentes, y soluciones de grado empresarial (Enterprise) con capacidades de personalización, contratos de nivel de servicio (SLA) y soporte técnico dedicado.[16] La implementación técnica requiere la creación de estos planes de suscripción a través de las interfaces de programación de aplicaciones (APIs) del procesador de pagos elegido, definiendo de manera inmutable en el sistema el nombre del plan, la frecuencia de cobro y los montos correspondientes.[18]

La siguiente tabla presenta un modelo arquetípico de estructuración de niveles de suscripción para la plataforma SaaS de automatización con inteligencia artificial, ilustrando la segmentación estratégica de características y límites de uso (quotas):

Métrica de Funcionalidad	**Nivel Básico (Plan Basic)**	**Nivel Profesional (Plan Pro)**	**Nivel Corporativo (Plan Enterprise)**
Precio Mensual de Referencia	$9	$29	$99+ (Cotización Personalizada)
Límite de Proyectos Activos	Hasta 5 proyectos	Capacidad Ilimitada	Capacidad Ilimitada

Límite de Usuarios Concurrentes	1 usuario (Propietario)	Hasta 10 usuarios	Usuarios Ilimitados
Cuota de Almacenamiento	1 GB de retención	10 GB de retención	100 GB de retención o más
Nivel de Analítica y Reportes	Informes visuales estándar	Informes avanzados y exportables en CSV	Cuadros de mando (dashboards) a medida
Nivel de Soporte Técnico	Vía correo electrónico (SLA 48h)	Soporte Prioritario (SLA 24h)	Gerente de cuenta dedicado (SLA 4h)

Lógica Transaccional y Prevención de Fricción

El backend de la aplicación, el servidor central que procesa la lógica, debe gestionar el ciclo de vida completo de la suscripción de manera autónoma. Esto requiere la configuración de puntos finales (endpoints) que actúen como receptores de notificaciones web (webhooks) provenientes de la infraestructura de Mercado Pago.[16] Estos webhooks informan al sistema cuando un pago ha sido aprobado, rechazado o cuando una suscripción ha sido cancelada, permitiendo al sistema actualizar los niveles de acceso del usuario, las cuotas de almacenamiento y los permisos en la base de datos en tiempo real, sin intervención administrativa humana.

Resulta crítico y mandatorio implementar lógicas de reintento automático para pagos rechazados (debido a fondos insuficientes o tarjetas expiradas), estableciendo períodos de gracia (dunning management) que alerten al usuario sin interrumpir abruptamente el servicio, protegiendo así la relación comercial.[16] De manera complementaria, el sistema debe aprovechar las herramientas nativas de prevención de fraudes y verificación de identidad del cliente que provee la pasarela de pagos para mitigar contracargos y transacciones maliciosas que puedan perjudicar el historial crediticio de la empresa operadora del SaaS.[17]

Fase 4: El Paradigma Agéntico y la Preparación del Entorno Google Antigravity

Habiendo cimentado la arquitectura comercial, legal y financiera, el proyecto ingresa en su fase

de desarrollo y ensamblaje técnico. Históricamente, este proceso requería la escritura manual de miles de líneas de código en editores estáticos. Hoy, la inteligencia artificial ha cambiado intrínsecamente la forma en que el software se diseña, se construye y se mantiene.[1] Si herramientas como Visual Studio Code (VS Code) fueron el símbolo del desarrollo de software moderno, Google Antigravity IDE representa la frontera indiscutida de la próxima generación: un entorno capaz de comprender holísticamente al desarrollador, anticipar sus intenciones sistémicas y colaborar en la creación de soluciones de forma proactiva.[19]

El concepto original de "Antigravity", que en el lenguaje de programación Python derivaba de una broma documentada sobre la facilidad absurda de importar librerías complejas, ha evolucionado en este contexto para convertirse en una metáfora literal: la capacidad de elevar a los equipos de ingeniería por encima de las limitaciones cognitivas y operativas actuales.[19] Antigravity no es un mero asistente de autocompletado de código tipo GitHub Copilot; es una plataforma de desarrollo diseñada desde sus cimientos bajo un enfoque centrado en el agente (agent-first).[1] Presupone que la inteligencia artificial es un actor autónomo con la capacidad de planificar estrategias de desarrollo, ejecutar operaciones de terminal, validar resultados mediante pruebas automatizadas e iterar sobre tareas complejas.[2]

El Centro de Control de Misiones y la Orquestación Paralela

La interfaz de Antigravity bifurca la experiencia tradicional en dos componentes principales: el Editor clásico y el "Agent Manager" o Centro de Control de Misiones.[2] A diferencia de un árbol de archivos convencional, este panel de control está diseñado para engendrar (spawn), monitorear y gobernar múltiples agentes asíncronos distribuidos en tareas independientes.[2] Un desarrollador que asume el rol de arquitecto puede asignar a un agente la reestructuración de la base de datos de PostgreSQL para acomodar los niveles de suscripción, mientras que simultáneamente comisiona a un segundo agente la refactorización de los componentes del frontend de Vue 2 a Tailwind CSS, y a un tercero la generación de la documentación técnica.[2] Esta capacidad multiplica el rendimiento exponencialmente, rompiendo el cuello de botella de la programación lineal.[2]

Para el motor cognitivo subyacente, Antigravity soporta la selección de múltiples modelos de frontera (frontier models), destacándose Gemini 3 Pro como el modelo predeterminado optimizado para el razonamiento de múltiples pasos y tareas de desarrollo autónomo.[22] De igual manera, la plataforma ofrece compatibilidad nativa con modelos de la competencia directa como Claude Sonnet 4.5 y GPT-4 de OpenAI, permitiendo al arquitecto seleccionar la inteligencia más adecuada para tareas específicas.[22] El entorno clasifica la interacción en dos modalidades de pensamiento: el Modo de Planificación (Planning Mode), empleado para investigación profunda y tareas arquitectónicas complejas donde el agente organiza el trabajo en artefactos y grupos de tareas consumiendo un mayor presupuesto de inferencia; y el Modo Rápido (Fast Mode), destinado a tareas localizadas y simples como la ejecución de comandos bash o el renombramiento semántico de variables.[2]

Políticas de Seguridad y Prevención de Catástrofes Agénticas

Otorgar a agentes algorítmicos la capacidad de ejecutar código y modificar sistemas introduce vectores de riesgo sin precedentes. La literatura reciente sobre la "Economía Agéntica" y los protocolos de mensajería de agentes advierte sobre incidentes catastróficos documentados, como agentes autónomos eliminando volúmenes completos de Google Drive o bases de datos de producción enteras durante períodos de congelamiento de código (code freeze) debido a una identidad sin restricciones de alcance (scope).[23] Antigravity aborda esta vulnerabilidad de ejecución persistente obligando al usuario a configurar políticas de autonomía estrictas durante el asistente de instalación inicial.[2]

Al configurar el entorno para el desarrollo del SaaS, el arquitecto debe definir mandatoriamente:

1. **Política de Ejecución de Terminal (Terminal Execution Policy):** Controla la fricción entre la decisión del agente y el sistema operativo anfitrión. Puede configurarse en modo *Apagado* (requiere permiso explícito para cada comando), modo *Automático* (el agente evalúa el riesgo y decide si solicita revisión), o modo *Turbo* (ejecución continua).[2] Para un proyecto crítico, se recomienda el uso de configuraciones personalizadas que operen sobre Listas de Permisos (Allow Lists) y Listas de Denegación (Deny Lists) para comandos de consola.[2]
2. **Política de Revisión (Review Policy):** Determina el nivel de supervisión humana sobre los artefactos de planificación generados antes de que se transformen en código ejecutable. La configuración recomendada es el *Desarrollo Asistido por Agentes*, el cual proporciona un equilibrio simbiótico entre la autonomía algorítmica y la auditoría humana, previniendo alteraciones estructurales no deseadas en el código base.[2]
3. **Seguridad del Navegador (Browser Security):** Restringe las capacidades del agente para interactuar con contenido web externo mediante el uso del Protocolo de Contexto de Modelos (MCP), mitigando el riesgo de exfiltración de datos o interacción con APIs no autorizadas.[2]

Fase 5: Metodología Paso a Paso para la Inyección en Antigravity

La fase crítica del proyecto consiste en trasladar toda la planificación estratégica comercial detallada en las fases anteriores hacia el código fuente del producto. En el paradigma tradicional, esto implicaría abrir archivos en blanco y tipear la lógica. En el entorno de Antigravity, este proceso se realiza mediante la "inyección" de contexto y metodologías estructuradas que los agentes usarán para programar la aplicación por nosotros.[2]

La arquitectura de inyección en Google Antigravity se cimienta sobre tres pilares conceptuales que deben ser cuidadosamente configurados en el directorio raíz del espacio de trabajo del proyecto: Reglas (Rules), Flujos de Trabajo (Workflows) y Habilidades (Skills).[25] Comprender la

diferenciación ontológica entre estos tres mecanismos es vital para evitar la saturación de la ventana de contexto del modelo de inteligencia artificial y garantizar que el agente actúe con extrema precisión.[25]

Paso 1: Inyección de Reglas (Rules) - Los Rieles de Seguridad Pasivos

Las Reglas son directrices de naturaleza pasiva que funcionan como barandillas de contención permanentes (always-on guardrails).[25] Estos documentos, alojados estrictamente dentro del directorio .agents/rules/, se inyectan de forma perpetua en el mensaje del sistema (system prompt) que gobierna el comportamiento global del agente en todo el espacio de trabajo.[26] Su propósito primordial no es iniciar una acción, sino restringir el *cómo* se ejecuta cualquier tarea encomendada, imponiendo estándares inquebrantables de codificación y cumplimiento de normas.[26]

Para garantizar que el SaaS cumpla con los requisitos operativos y legales del mercado argentino, el arquitecto debe crear archivos Markdown dentro de esta carpeta con las siguientes directrices estructurales:

1. **Regla de Cumplimiento Impositivo (.agents/rules/afip_compliance.md):** Se redacta una instrucción en lenguaje natural obligando al agente a que, en cualquier momento que se genere código relacionado con facturación o procesamiento financiero, asuma que la entidad operadora es un Responsable Inscripto. Se le instruye a utilizar librerías de precisión de doble punto flotante para prevenir discrepancias de redondeo en el cálculo del IVA y a validar rigurosamente la estructura y módulo once de la Clave Única de Identificación Tributaria (CUIT) en todos los formularios de entrada de datos.[13]
2. **Regla de Arquitectura Frontend (.agents/rules/frontend_architecture.md):** Considerando que el proyecto busca una alta escalabilidad y un desarrollo ágil, se inyecta una regla que dictamina el uso sistemático del entorno de trabajo Tailwind CSS en combinación con el modo estricto de TypeScript.[21] Esta regla asegura que, al delegar la creación de modales de pago o paneles de control, el agente no introduzca librerías visuales obsoletas o vulnerables, manteniendo la homogeneidad estilística de la interfaz.[21]
3. **Regla de Manejo de Secretos (.agents/rules/security_secrets.md):** Se establece un mandato global de ciberseguridad que prohíbe terminantemente comprometer (commit) claves de API de bases de datos o tokens de acceso a repositorios públicos, forzando la utilización de variables de entorno locales (.env).[26]

La composicionalidad de Antigravity permite que estas directivas globales interactúen con otras capas de abstracción; por ejemplo, una regla puede forzar la utilización obligatoria de una "Habilidad" específica cuando detecta manipulaciones en el esquema de la base de datos de los usuarios, previniendo la eliminación accidental de registros.[25]

Paso 2: Inyección de Habilidades (Skills) - El Cerebro Operativo Dinámico

Mientras que las Reglas son pasivas, las Habilidades (Skills) ocupan un punto intermedio dinámico. Representan el componente más sofisticado del ecosistema, operando como paquetes encapsulados de conocimiento experto que el agente invoca de manera autónoma cuando el modelo detecta la intención del usuario o la necesidad técnica de la tarea en curso.[25] Esto significa que el arquitecto no necesita ordenar explícitamente el uso de una habilidad; el sistema dota al agente temporalmente de esta experiencia para interactuar con herramientas, ejecutar scripts y procesar lógicas complejas, evitando saturar el contexto general del modelo con información irrelevante cuando no se está ejecutando esa tarea específica.[25]

La arquitectura de directorios exige alojar estas habilidades en <workspace-root>/.agents/skills/<skill-folder>/ para restringir su alcance al proyecto actual, o en ~/.gemini/antigravity/skills/ si se desea que la habilidad esté disponible globalmente para el sistema operativo.[27]

La anatomía formal de una Habilidad requiere imperativamente la creación de un archivo SKILL.md dentro de su subdirectorio correspondiente, el cual incluye metadatos de configuración YAML en su cabecera para definir su nombre y descripción de uso.[27] Además, la carpeta de la habilidad puede contener subdirectorios opcionales pero altamente recomendados: scripts/ para la lógica ejecutora auxiliar en lenguajes como Python o Bash, examples/ para proveer implementaciones de referencia, y resources/ para plantillas estructurales.[27]

El cuerpo del archivo SKILL.md debe estructurarse mediante ingeniería de instrucciones (prompt engineering) precisa.[25] Debe incluir: un *Objetivo* (Goal) claro, *Instrucciones* (Instructions) procedimentales paso a paso, *Ejemplos* (Examples) basados en aprendizaje de pocos disparos (few-shot prompting) para calibrar el resultado esperado, y *Restricciones* (Constraints) de lo que no se debe realizar.[25]

Inyección Práctica: Integración de la Suscripción de Mercado Pago

Para materializar la arquitectura financiera diseñada en la Fase 3, el arquitecto procede a inyectar una habilidad dedicada a la construcción y mantenimiento de la pasarela de cobro recurrente.

1. Se crea el directorio .agents/skills/mercadopago-subscriptions/.[27]
2. Se elabora el documento SKILL.md con la siguiente cabecera YAML:
   YAML
   ```
   ---
   name: mercadopago-subscriptions
   description: Habilidad especializada en la creación, gestión e integración técnica de planes de suscripción (Basic, Pro, Enterprise) utilizando el SDK de Mercado Pago Argentina. Se activa cuando el usuario menciona monetización, cobros recurrentes o pasarelas de pago.
   ---
   ```

3. En el cuerpo del documento, el arquitecto instruye al agente a identificar los parámetros del nivel de suscripción solicitados (precio, límites de proyectos) basándose en la petición en lenguaje natural.[26]
4. Se instruye al agente para que ejecute un script auxiliar preconstruido alojado en scripts/scaffold_mp.py utilizando el entorno de ejecución (Runtime) local, el cual inicializa los archivos del controlador backend y las rutas de la API.[26]
5. El documento SKILL.md dirige la atención del agente a la carpeta examples/webhook_handler_reference.ts, asegurando que el código que genere para procesar los avisos de la pasarela posea el formato criptográfico correcto para validar el origen de los datos, evitando vulnerabilidades de falsificación de peticiones (spoofing).[18]
6. Finalmente, la habilidad detalla la restricción absoluta de almacenar números de tarjetas de crédito en la base de datos de la plataforma SaaS, forzando la utilización exclusiva de los identificadores seguros proporcionados por los puntos finales de la API financiera.[17]

Cuando el arquitecto interactúa con el Centro de Control de Misiones y solicita: "Construye el portal de configuración de pagos para que el usuario contrate el plan Profesional de 29 dólares", el agente de Antigravity detecta la intención, carga en memoria la habilidad mercadopago-subscriptions, ejecuta el script de andamiaje, lee la documentación de referencia, programa el panel frontend utilizando los componentes de Tailwind CSS estipulados por la Regla Global, y conecta la lógica del backend para emitir la solicitud de cobro recurrente.[18]

Inyección Práctica: Funcionalidades Core de IA para el MVP

La funcionalidad principal que da valor al SaaS (las aplicaciones de inteligencia artificial para la PyME) se inyecta utilizando el mismo patrón metodológico. Si el producto estrella es un analizador automatizado de archivos de ventas, se crea la habilidad .agents/skills/csv-sales-analyzer/.

Esta habilidad empaqueta instrucciones para que el agente construya conductos (pipelines) de procesamiento de datos.[7] En lugar de escribir el código analítico manualmente, el arquitecto provee en examples/ modelos matemáticos estadísticos estándar. Cuando se le ordena al agente implementar la funcionalidad de "Análisis de Demanda", la habilidad le instruye a utilizar librerías estandarizadas (como Pandas en Python o sus equivalentes en Node.js) para clasificar las imágenes de catálogos o generar reportes de rendimiento comercial, traduciendo requerimientos conceptuales en infraestructura de código robusta de manera autónoma.[5] Adicionalmente, se pueden integrar Servidores MCP (Model Context Protocol) desde la tienda de extensiones para dotar a estas habilidades de acceso seguro a herramientas de bases de datos externas o servicios en la nube nativos, ampliando el espectro de acción del agente.[2]

Paso 3: Inyección de Flujos de Trabajo (Workflows) - Macros Operativos

A diferencia de las Habilidades, los Flujos de Trabajo (Workflows) son secuencias activas que no se disparan por inferencia del agente, sino que son macros de nivel superior desencadenados explícitamente por comandos del usuario (por ejemplo, /test o /deploy).[25] Residen en el directorio .agents/workflows/ y poseen la capacidad arquitectónica de orquestar múltiples Habilidades de manera secuencial para completar procesos iterativos extensos y repetitivos.[25]

La inyección de flujos de trabajo en el proyecto SaaS es crítica para democratizar las operaciones de desarrollo (DevOps) y estandarizar la garantía de calidad del software.[29] El desarrollador inyectará un archivo .agents/workflows/deploy-pipeline.md. Al invocar el comando /deploy-pipeline en la terminal del agente, el sistema Antigravity ejecutará un guión pregrabado que instruirá a la fuerza laboral digital a:

1. Examinar el código base en busca de vulnerabilidades de seguridad y variables de entorno expuestas.
2. Invocar una habilidad de pruebas unitarias para simular transacciones de Mercado Pago fallidas y verificar que el sistema de reintentos funcione según lo diseñado.
3. Auditar la interfaz gráfica responsiva del panel de usuario para asegurar que la visualización de datos operará correctamente en dispositivos móviles.
4. Preparar el empaquetado del software para su migración a servidores en la nube.

Estos flujos reducen dramáticamente la carga cognitiva del equipo humano, asegurando que los procedimientos estándar de la industria se sigan rigurosamente sin depender de la memoria de los operadores.[29]

Fase 6: Transición a Producción y Ejecución Agéntica en la Nube

Una vez que el desarrollo local dentro del entorno de Antigravity ha alcanzado la madurez operativa y ha superado exitosamente los flujos de trabajo de prueba, el ciclo de vida del producto exige su transición hacia un entorno de producción accesible a través de internet.[30] Este paso es fundamental, ya que mientras un entorno local depende del estado operativo y el poder de cómputo de la máquina del desarrollador, una plataforma SaaS orientada al mercado debe operar ininterrumpidamente, procesar peticiones web de múltiples clientes simultáneos y mantener tiempos de latencia ínfimos, garantizando una alta disponibilidad.[31]

Migración del Código a Infraestructura en la Nube

Para facilitar la transición fluida de aplicaciones, herramientas integradas como Firebase Studio proporcionan procesos automatizados de migración y empaquetado. Esta migración utiliza los propios agentes de Antigravity para transformar la estructura del proyecto local hacia arquitecturas compatibles con el alojamiento en la nube de forma autónoma.[30] El arquitecto, desde la interfaz de Antigravity, puede utilizar modelos de lenguaje optimizados para velocidad

y eficiencia en el manejo de altos volúmenes de datos, como Gemini Flash, en conjunto con instrucciones especializadas (prompts) como @fbs-to-agy-export.[30] El agente inicia el proceso de exportación local, previsualización de la aplicación y posterior publicación aplicando las mejores prácticas operativas para bases de datos no relacionales y sistemas de autenticación administrados.[30]

Alternativamente, foros de discusión técnica e intercambios entre desarrolladores pioneros en la comunidad de Antigravity evidencian la viabilidad y alta eficiencia de delegar a los agentes el despliegue del proyecto hacia infraestructuras combinadas, como el alojamiento de los componentes visuales interactivos en proveedores edge computing como Vercel, acoplado con bases de datos relacionales administradas como Supabase.[8] Al instruir a los modelos Claude o Gemini integrados en Antigravity con los comandos de interfaz de línea (CLI) correspondientes (por ejemplo, utilizando el gcloud cli para infraestructura de Google Cloud), los agentes asumen el control de la consola, resuelven dependencias, ejecutan la compilación del código fuente y transfieren los artefactos finales al entorno en vivo.[33] Este nivel de automatización elimina la barrera técnica que tradicionalmente impedía a creadores de interfaces o analistas de negocios lanzar plataformas sofisticadas al mercado.[33]

Despliegue de Agentes para Ejecución 24/7 (Serverless Agentic Workflows)

Un aspecto distintivo en la arquitectura de un SaaS de inteligencia artificial moderna radica en que la lógica del negocio no solo reside en la presentación visual, sino en el procesamiento en segundo plano que los agentes realizan para el usuario final (por ejemplo, el procesamiento de lotes masivos de contratos o la inferencia analítica de datos).[28] Si estos procesos agénticos se ejecutan dependientes de la infraestructura local del desarrollador, el SaaS colapsaría al apagar el equipo.

Por consiguiente, la orquestación requiere ir más allá del alojamiento web convencional para incorporar plataformas de ejecución computacional sin servidor (serverless) altamente especializadas, tales como Modal.com y Trigger.dev.[31] Estas infraestructuras permiten extraer los flujos de trabajo agénticos (agentic workflows) desarrollados en Antigravity y encapsularlos en contenedores virtuales persistentes que operan en los servidores de la nube.[31] Esta arquitectura garantiza un tiempo de actividad del cien por ciento (24/7 uptime).[31]

A través de esta integración, el panel de control web (dashboard front-end) construido y subido previamente actúa únicamente como una interfaz de activación.[31] Cuando el cliente PyME solicita desde la plataforma web el procesamiento de un archivo, la aplicación emite una señal a través de una API hacia plataformas como Modal.[31] Allí, instancias remotas y equivalentes a los agentes de Antigravity reciben la información, ejecutan el análisis de datos prolongado de manera asíncrona sin bloquear la experiencia del usuario, y retornan los resultados procesados—por ejemplo, permitiendo al usuario descargar un reporte ejecutivo

compilado, añadiendo funciones iterativas de experiencia de usuario como el "Download CSV".[31] La gestión del control de versiones y el historial de ejecución se centraliza en estas plataformas especializadas, asegurando la trazabilidad de las inferencias y permitiendo al arquitecto escalar la operación del SaaS de IA para miles de clientes simultáneamente, cimentando un modelo de agencia o servicio verdaderamente automatizado e inteligente.[31]

Conclusión

El proceso de llevar un proyecto tecnológico de automatización desde su estado conceptual hasta su inserción en el complejo mercado argentino trasciende la mera programación secuencial de líneas de código. Se constituye como un esfuerzo multidisciplinario y orquestado que entrelaza la validación temprana de un Producto Mínimo Viable, la comprensión rigurosa del cumplimiento fiscal tributario impuesto por regulaciones gubernamentales, la estructuración legal de tratamiento de datos, y la consolidación de una arquitectura financiera de pagos recurrentes ininterrumpida.

La adopción de Google Antigravity como ecosistema central materializa la transición definitiva hacia la economía agéntica, empoderando al creador para actuar como un gerente de sistemas cognitivos. Al estructurar metodológicamente el entorno de desarrollo mediante Reglas globales pasivas, Flujos de Trabajo imperativos y Habilidades dinámicas altamente parametrizadas, el desarrollador mitiga riesgos operacionales catastróficos y eleva exponencialmente su capacidad de producción. La inyección sistémica de módulos de extrema complejidad—desde la lógica condicional de comprobantes electrónicos hasta la integración de APIs financieras y el despliegue de infraestructura en la nube para el procesamiento asíncrono permanente—se convierte en un ejercicio elegante de diseño arquitectónico algorítmico. Esta metodología no solo reduce dramáticamente el tiempo de salida al mercado (time-to-market), sino que garantiza la robustez estructural de la plataforma, alineando el desarrollo del software con los objetivos inalienables de escalabilidad, eficiencia operativa comercial y un retorno de inversión sostenido para sus usuarios finales.

Obras citadas

1. Agent-Powered Development with Google Antigravity - Jan 26th 2026 - YouTube, fecha de acceso: marzo 22, 2026, https://www.youtube.com/watch?v=FqoT_WdbO-I
2. Tutorial : Getting Started with Google Antigravity | by Romin Irani ..., fecha de acceso: marzo 22, 2026, https://medium.com/google-cloud/tutorial-getting-started-with-google-antigravity-b5cc74c103c2
3. Checklist SaaS: 13 pasos para prelanzamiento éxito, fecha de acceso: marzo 22, 2026, https://blog.payproglobal.com/es/crafting-a-comprehensive-saas-product-checklist
4. How to prepare a SaaS for launch - Checklist to avoid mistakes - YouTube, fecha

de acceso: marzo 22, 2026, https://www.youtube.com/watch?v=bCVRhDb4cg8
5. Inteligencia artificial para pymes argentinas: Guía práctica - Neowyze, fecha de acceso: marzo 22, 2026, https://neowyze.tech/blog/inteligencia-artificial-para-pymes-argentina
6. Diplomatura en Inteligencia Artificial Práctica, Automatización y Productividad para el Trabajo y los Negocios - Universidad de Belgrano, fecha de acceso: marzo 22, 2026, https://ub.edu.ar/sites/default/files/ia_aplicada_automatizaci%C3%B3n_y_productividad_final.pdf
7. Antigravity: La IA Que Hace en Segundos Lo Que Un Programador Tarda Horas - YouTube, fecha de acceso: marzo 22, 2026, https://m.youtube.com/watch?v=ELIE_un1Rn4
8. ¡Transforma tu aprendizaje con nuestro taller de Antigravity! #shorts - YouTube, fecha de acceso: marzo 22, 2026, https://www.youtube.com/shorts/s3sXbBilrpo
9. Automatización en PYMEs Argentina 2025: Guía paso a paso - Nexo Estelar, fecha de acceso: marzo 22, 2026, https://nexoestelar.com/automatizacion-procesos-pymes-argentina-2025/
10. Cómo preparar el lanzamiento de un SaaS sin sustos: seguridad, cobros, analítica y legal en orden - Bolmia, fecha de acceso: marzo 22, 2026, https://bolmia.com/blog/desarrollo-saas-a-medida-checklist-lanzamiento-seguridad-billing-analitica-legal/
11. Guía completa de marketing para SaaS: estrategias para el éxito - Marketic, fecha de acceso: marzo 22, 2026, https://marketic.com.ar/marketing-saas/
12. Monotributo vs Responsable Inscripto ¿Cuál de los 2 Conviene más? - YouTube, fecha de acceso: marzo 22, 2026, https://www.youtube.com/watch?v=ds_pnEVshbY
13. Régimen general | Facturación - ARCA, fecha de acceso: marzo 22, 2026, https://www.afip.gob.ar/facturacion/regimen-general/
14. Responsable Inscripto 2026: Tabla, cuánto se paga y guía completa | Blog Contablix, fecha de acceso: marzo 22, 2026, https://contablix.ar/blog/responsable-inscripto-guia-definitiva-impuestos/
15. Ley simple: Recomendaciones para el uso de Inteligencia Artificial - Argentina.gob.ar, fecha de acceso: marzo 22, 2026, https://www.argentina.gob.ar/justicia/derechofacil/leysimple/educacion-ciencia-cultura/recomendaciones-para-el-uso-de
16. Cómo gestionar las suscripciones SaaS: Una guía paso a paso - PayPro Global, fecha de acceso: marzo 22, 2026, https://payproglobal.com/es/guias/gestionar-suscripciones-saas/
17. Resumen - Suscripciones - Mercado Pago Developers, fecha de acceso: marzo 22, 2026, https://www.mercadopago.com.ar/developers/es/docs/subscriptions/overview
18. ¿Cómo cobrar una suscripción de forma recurrente? - Mercado Pago, fecha de acceso: marzo 22, 2026, https://www.mercadopago.com.ar/ayuda/19543
19. Antigravity IDE: la nueva generación de editores impulsados por IA - DEV Community, fecha de acceso: marzo 22, 2026,

https://dev.to/vicodev/antigravity-ide-la-nueva-generacion-de-editores-impulsados-por-ia-56m4
20. Getting Started with Google Antigravity, fecha de acceso: marzo 22, 2026, https://codelabs.developers.google.com/getting-started-google-antigravity
21. Antigravity y el nuevo paradigma del desarrollo - Amoba Software, fecha de acceso: marzo 22, 2026, https://amobasoftware.com/blog/post/103/antigravity-y-el-nuevo-paradigma-del-desarrollo
22. How to Set Up and Use Google Antigravity - Codecademy, fecha de acceso: marzo 22, 2026, https://www.codecademy.com/article/how-to-set-up-and-use-google-antigravity
23. ECONOMÍA AGÉNTICA Frameworks para gobernar el trabajo de agentes - ResearchGate, fecha de acceso: marzo 22, 2026, https://www.researchgate.net/publication/402252305_ECONOMIA_AGENTICA_Frameworks_para_gobernar_el_trabajo_de_agentes
24. Forced Descent: Google Antigravity Persistent Code Execution Vulnerability - Mindgard AI, fecha de acceso: marzo 22, 2026, https://mindgard.ai/blog/google-antigravity-persistent-code-execution-vulnerability
25. Authoring Google Antigravity Skills, fecha de acceso: marzo 22, 2026, https://codelabs.developers.google.com/getting-started-with-antigravity-skills
26. Tutorial : Getting Started with Google Antigravity Skills, fecha de acceso: marzo 22, 2026, https://medium.com/google-cloud/tutorial-getting-started-with-antigravity-skills-864041811e0d
27. Agent Skills - Google Antigravity Documentation, fecha de acceso: marzo 22, 2026, https://antigravity.google/docs/skills
28. ¡Descubre Proyectos Innovadores con Antigravity! #shorts - YouTube, fecha de acceso: marzo 22, 2026, https://www.youtube.com/shorts/muwHutePypY
29. Antigravity ha Destrozado las Pruebas Técnicas para Programadores... y si no te lo crees, mira esto. - YouTube, fecha de acceso: marzo 22, 2026, https://www.youtube.com/watch?v=_E5mehnlyfw
30. Firebase Studio sunset and project migration - Google, fecha de acceso: marzo 22, 2026, https://firebase.google.com/docs/studio/migrating-project
31. The Easiest Way to Deploy AntiGravity Agentic Workflows in 2026 - YouTube, fecha de acceso: marzo 22, 2026, https://www.youtube.com/watch?v=dprcTRFHPHY
32. Firebase Studios Migration - Google Antigravity Documentation, fecha de acceso: marzo 22, 2026, https://antigravity.google/docs/firebase-studio-migration
33. How to actually deploy web apps after creating them in Antigravity? - Reddit, fecha de acceso: marzo 22, 2026, https://www.reddit.com/r/google_antigravity/comments/1pu35w9/how_to_actually_deploy_web_apps_after_creating/